CSE 150: Introduction to Computer Networks

Chris Parsa

Ch 3 Part 2

0

Assignments

- Reading: Entire Chapter 3 sections 1,2,3,4,5,6,7
- Chapter 3 Practice
  - "R" Problems Sections 3.1-3.4: R2 (think about letter-writing example in book/class), R3, R4, R5, R6, R7, R8, R9, R10, R11
  - "R" Problems Sections 3.5-3.7: sections 3.5 - 3.7 (R14, R15, R16, R17, R18 and R19)
- Lab 5
- Group Assignment 4

1

1

Lecture this week and next:

- Intro to Transport Layer ✓
- Internet Checksum ✓
- Section 3.4: rdt 1.0, 2.0, 2.1 ✓
- GoBackN and Selective Repeat
- Then TCP

2

Warmup

Talk to your neighbor:

With the rdt X protocols, how many packets is the Sender transmitting at any given time?

Justify your answer by looking at the rdt 3.0 state diagram.

4

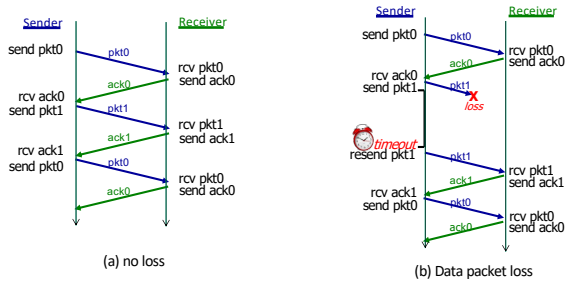
Continue with Chapter 3

A different view of rdt 3.0

A few more "learning" protocols" for Reliable Data Transfer

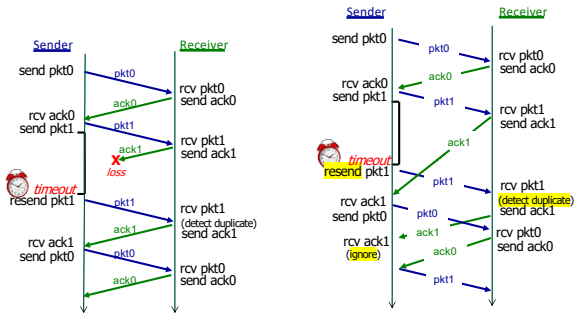
5

rdt3.0 in action – a different view



6

rdt3.0 in action



(c) ACK loss

(d) Premature timeout/ delayed ACK
(sanity check: rdt 3.0 state diagram)

7

Performance of rdt x

9

Performance of Stop and Wait protocols?

- One packet at a time!
- Review: Throughput (avg and instantaneous), Delay and Link utilization (AKA intensity)

Goal: To fully utilize available bandwidth,

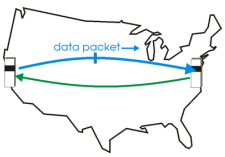
- try to match transmission rate to what network can offer (link utilization)

10

Use the Link!

Link Utilization:

- Fraction of time sender is doing useful work
  - i.e., transmitting
- Average traffic over a link expressed as a percentage of the total link capacity



(a) a stop-and-wait protocol in operation

AKA traffic intensity

11

Performance of rdt3.0 (stop-and-wait)

Example: 1 Gbps link, 15 msec prop. delay, 8000 bit packet

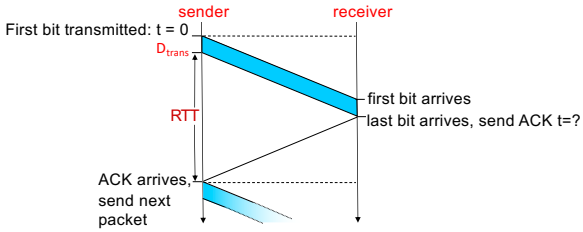
Time to transmit a single packet into channel:

$$D_{trans} = \frac{L}{R} = \frac{8000\ bits}{10^9\ bits/sec} = 8\ \mu s$$

- Propagation… one way prop delay = 15msec
- What is the Sender doing now?

12

rdt3.0: stop-and-wait operation



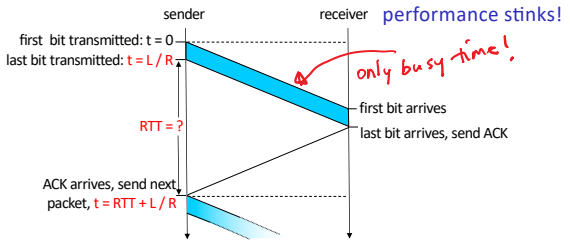
Next packet transmitted at time $t = D_{trans} + RTT$

15

rdt3.0: Utilization

8000 bit pkt transmitted every 30.008 msec → 267 kbps transmission rate over 1 Gbps link

performance stinks!



- ACK arrives at t= RTT + L/R =~ 30.008msec

L=8000bits
RTT=30msec (why ?)
R=1Gbps
D_{trans} = 8 µs = .008ms

$$U_{sender} = \frac{work}{time} = \frac{L/R}{RTT + L/R} = \frac{.008\ ms}{30.008\ ms} = 0.00027 < 1\%$$

Rdt3.0 works but performance stinks!

16

How can we improve performance?

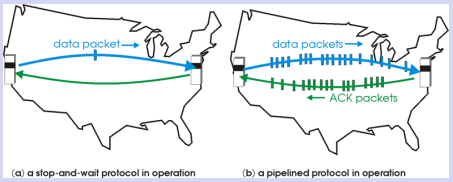
- Network as storage
- BWDP: Bandwidth delay product
  - In previous example, how many packets fit on the link?

18

Pipelined protocols (ARQ Protocols)

Send multiple, "in-flight", yet-to-be-acknowledged packets

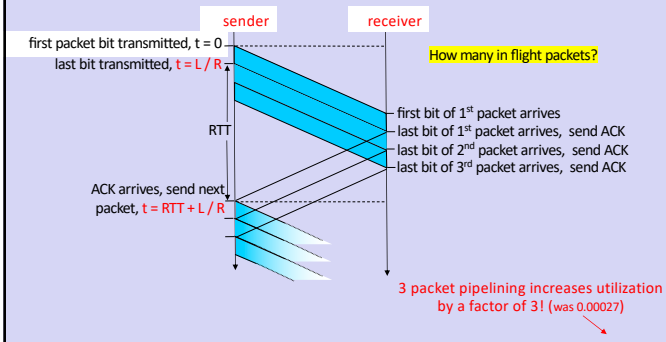
- Range of sequence numbers must be increased from 0,1
- Is buffering needed at sender and receiver →why?



- Two generic forms of pipelined protocols: *Go-back-N, Selective Repeat*

20

Pipelining: increase utilization



3 packet pipelining increases utilization by a factor of 3! (was 0.00027)

L=8000bits
RTT=30msec
R=1Gbps
D_{trans} = 8 µs = .008ms

$$U_{sender} = \frac{work}{time} = \frac{3L/R}{RTT + L/R} = \frac{.0024\ ms}{30.008\ ms} = 0.00081$$

21

GoBackN Protocol

23

Go-Back-N: Sender

- "WINDOW" of up to N "packets in flight" AKA unACKed packets
- Each packet labeled with SN in pkt header (increasing numbers)

- *Cumulative ACK:*
  - ACK(*n*) - all packets received are in-order up to and including SN = *n*
- If ACK(*n*) > `send_base`: update `send_base` = *n+1*
  - Window slides over
- Set timer for oldest in-flight packet
- *timeout(n):* retransmit all packet in window (includes n)

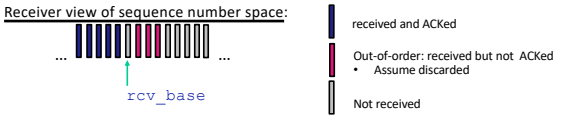
25

Cumulative ACKs

26

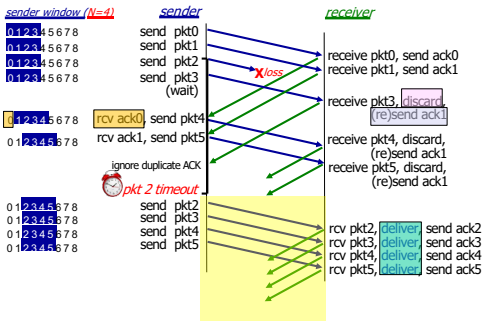
Go-Back-N: Receiver

- *Cumulative ACK* : sent for each packet received (ACK # of highest *in-order* seq # received)
  - only remember `rcv_base`
  - may generate duplicate ACKs
- If received packet is OUT-OF-ORDER:
  - Discard or buffer → an implementation decision
    - *In our GBN example →Assume discard out of order packets*
  - Send cumulative ACK (as above)
- If received packet is IN-ORDER:
  - Send ACK
  - Pass up to Application Layer



28

Go-Back-N in action



What do you notice?

29

What do you like about GBN?

What don't you like about GBN?

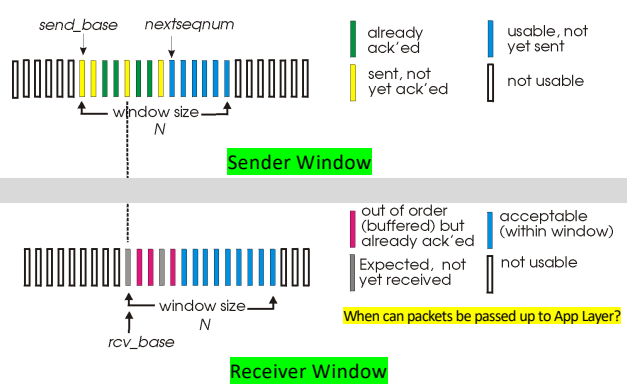
30

Selective Repeat Protocol

- "WINDOW" of up to N "packets in flight" AKA unACKed packets
- Each packet labeled with SN in pkt header (increasing numbers)
- Receiver *individually* acknowledges each received pkt
  - Selective ACK (echo the received SN)
  - Buffers out-of-order pkts
  - In-order delivery to upper layer (when?)
- Sender keeps track of ACKs from Receiver
  - Timer(n):
    - Sender sets a timer for **each** unACKed pkt
    - Retransmit **one packet** packet based on timer(n)

32

Selective Repeat: windows

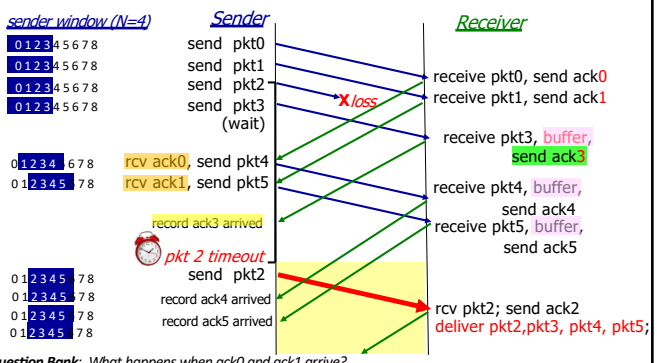


When can packets be passed up to App Layer?

33

Selective repeat in action

*How different from GoBackN?



Question Bank: *What happens when ack0 and ack1 arrive?*
Where is ack2?
What happens when pkt2 finally arrives at Receiver?
What happens when Sender receives ack2?

34

Pipelining Protocols - Comparison

Go-back-N: overview
- *Sender:*
- *Receiver:*
- *Retransmissions:*
- *Complexity of Protocol*

Selective Repeat: overview
- *Sender:*
- *Receiver:*
- *Retransmissions:*
- *Complexity of Protocol*

Practice - Make your list!!

36

Pipelining Protocols - Comparison

Go-back-N: overview
- *Sender:*
- *Receiver:*
- *Retransmissions:*
- *Complexity of Protocol*

Selective Repeat: overview
- *Sender:*
- *Receiver:*
- *Retransmissions:*
- *Complexity of Protocol*

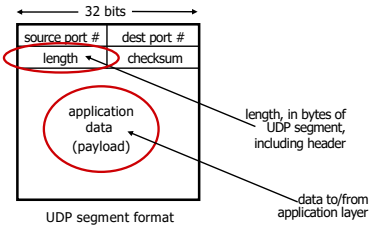
Practice - Make your list!!

38

UDP: User Datagram Protocol

39

UDP segment header



UDP segment format

8 bytes

Think about overhead of TCP vs UDP

40

UDP: User Datagram Protocol

- "no frills," "bare bones" Internet transport protocol
- "best effort" service, UDP segments may be:
  - lost
  - delivered out-of-order to app
- *connectionless:*
  - no handshaking between UDP sender, receiver
  - each UDP segment handled independently of others
- An option if reliable transfer needed over UDP
  - add reliability at application layer
  - add congestion control at application layer

Data Transmission
- UDP socket
- Datagram is carried in a single IP packet
- Limited to a maximum payload of 65,507 bytes for IPv4
- Usually "fragmented"

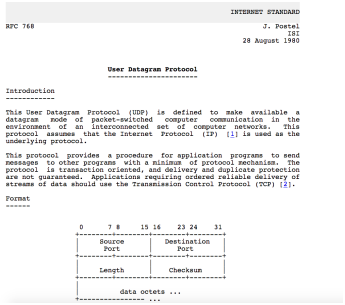
41

UDP: User Datagram Protocol

Why is there a UDP?

- no delay for connection establishment
- small header size
- simple: no connection state at sender, receiver
- no congestion control
  - UDP can blast away as fast as desired!
  - can function in the face of congestion
- Example use: multimedia, DNS

UDP: User Datagram Protocol [RFC 768]



INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

User Datagram Protocol

Introduction

This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format

```
 0      7 8     15 16    23 24    31
+--------+--------+--------+--------+
|     Source      |   Destination   |
|      Port       |      Port       |
+--------+--------+--------+--------+
|                 |                 |
|     Length      |    Checksum     |
+--------+--------+--------+--------+
|
|          data octets ...
+---------------- ...
```

Transport Layer: 3-44

Read it! -- just 2 pages

End Ch3 Part 2